The Journal of Open Source Education

FormEase: A Lightweight JavaScript Library for Intelligent Form Handling with Built-in Validation and Accessibility

Piyush Yadav [1]

1 Independent Developer, Open Source Contributor

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Submitted: 09 October 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

FormEase is a lightweight JavaScript library designed to simplify form handling in web applications by automating validation, data persistence, and accessibility features. Built with TypeScript, it provides a framework-agnostic solution that eliminates repetitive boilerplate code commonly required for form management. The library automatically detects input types and applies appropriate validation rules while maintaining full ARIA compliance for screen readers. With zero external dependencies and a bundle size under 13KB (minified), FormEase offers developers a production-ready tool that integrates seamlessly with vanilla JavaScript or modern frameworks like React, Vue, and Angular.

Statement of Need

Web forms remain a fundamental component of user interaction, yet implementing robust form validation and accessibility continues to be time-consuming and error-prone. Developers frequently write repetitive validation logic for common input types (email, URL, phone numbers) and struggle to maintain consistent accessibility standards across applications. Existing form libraries often come with significant overhead, tight framework coupling, or incomplete accessibility support.

FormEase addresses these challenges by providing an intelligent form handling solution that requires minimal configuration. Unlike traditional approaches where developers must explicitly define validation rules for every field, FormEase can automatically detect input types and apply sensible defaults when rules are not explicitly provided. The library handles common pain points such as debounced auto-saving to local storage, real-time validation feedback, and ARIA announcements for assistive technologies. By abstracting these complexities behind a simple API, FormEase allows developers to focus on application logic rather than form infrastructure.

The primary target audience includes web developers building interactive forms for data collection, user registration, surveys, and e-commerce checkout flows. The framework-agnostic design makes it particularly valuable for teams maintaining multiple projects with different technology stacks, as well as educators teaching web development fundamentals without framework-specific abstractions.

Key Features

FormEase provides the following capabilities:

- **Automatic Input Detection**: Identifies field types (email, URL, telephone, number, date) and applies appropriate validation without explicit configuration
- **Real-time Validation**: Validates input on both blur and keystroke events with customizable debouncing
- **Auto-save Functionality**: Persists form data to localStorage with configurable intervals to prevent data loss
- **Accessibility First**: Full ARIA support with live regions for screen reader announcements and keyboard navigation
- **Framework Agnostic**: Works with vanilla JavaScript and integrates cleanly with React, Vue, Angular, and other frameworks
- **TypeScript Support**: Comprehensive type definitions for improved developer experience and type safety
- **Zero Dependencies**: Self-contained implementation with no external runtime dependencies
- **Lightweight**: Minified bundle size of approximately 12.6KB, with tree-shaking support for ES modules
- **Custom Validators**: Extensible API for adding project-specific validation rules
- **Error Handling**: Structured error management with customizable error messages and positioning

Implementation

FormEase is implemented in TypeScript and compiles to multiple module formats (UMD, ESM, CommonJS) to support diverse deployment scenarios. The architecture separates concerns into distinct modules for validation, auto-save, and error handling, making the codebase maintainable and testable. The build process uses Rollup for efficient tree-shaking and module bundling.

The validation engine supports both synchronous and asynchronous validators, allowing for server-side validation checks. Input type detection leverages HTML5 input types (MDN Web Docs, 2025a) and pattern matching to infer validation requirements automatically. The accessibility layer implements WAI-ARIA 1.2 authoring practices (W3C Web Accessibility Initiative, 2021) and follows WCAG 2.1 guidelines (W3C Web Accessibility Initiative, 2018), ensuring compatibility with major screen readers including NVDA, JAWS, and VoiceOver.

The auto-save mechanism uses the browser's localStorage API with configurable debouncing to balance data persistence with performance. Form data is serialized to JSON and can be restored automatically when users return to incomplete forms, following best practices documented in MDN Web Docs (MDN Web Docs, 2025b).

Performance

Benchmark testing demonstrates that FormEase typically processes form validation events in under 1ms for common input types (email, URL, telephone, number), and auto-save operations typically complete within 3–5ms on average. The library maintains consistent performance even on low-end devices, making it suitable for mobile web applications and progressive web apps. Memory usage remains stable during extended sessions, with no memory leaks detected in long-running tests. The lightweight architecture and zero-dependency design contribute to minimal impact on application bundle size and startup time.

Example Usage

The following example demonstrates basic FormEase integration:

DRAFT

The Journal of Open Source Education

```javascript
import { FormEase } from '@piyushrajyadav/formease';

const form = new FormEase({
  form: '#contact-form',
  validation: {
    validateOnInput: true,
    validateOnBlur: true,
    rules: {
      email: {
        required: true,
        type: 'email',
        errorMessage: 'Please enter a valid email address'
      },
      name: {
        required: true,
        minLength: 2,
        errorMessage: 'Name must be at least 2 characters'
      },
      age: {
        type: 'number',
        min: 13,
        max: 120,
        errorMessage: 'Age must be between 13 and 120'
      }
    }
  },
  autoSave: {
    storageKey: 'contact-form-draft',
    debounceDelay: 2000
  }
});
```

For framework integration, FormEase provides lifecycle methods for cleanup and can be initialized within component mount hooks.

Testing and Quality Assurance

FormEase includes a comprehensive test suite with 53 unit and integration tests covering validation logic, auto-save functionality, error handling, and accessibility features. The library achieves full test coverage across all major code paths and is continuously tested against modern browsers (Chrome, Firefox, Safari, Edge).

Comparison with Existing Tools

Several JavaScript libraries address form handling, including Formik (Palmer, 2023), React Hook Form (Luo, 2023), and VeeValidate (Awad, 2023). However, these solutions are typically framework-specific and require developers to explicitly configure validation rules. FormEase distinguishes itself through automatic input type detection, framework independence, and comprehensive accessibility support built-in from the ground up. The zero-dependency architecture also makes FormEase suitable for projects where bundle size is critical.

DRAFT

Future Work

Planned enhancements for FormEase include the development of a plugin system for custom input types and validators, enabling developers to extend functionality without modifying core library code. Future versions will explore integration with IndexedDB for large-scale form data persistence, particularly beneficial for multi-step forms and complex data collection workflows. Additional work includes developer tooling such as visual form validation debugging utilities and enhanced error reporting for development environments. Community feedback will guide prioritization of features such as file upload validation, multi-language support for error messages, and integration with popular CSS frameworks.

Availability and Installation

FormEase is available through npm (Node Package Manager) and can be installed using:

```
npm install @piyushrajyadav/formease
```

The library is also available via CDN for direct browser usage. The source code, documentation, and interactive demo are hosted on GitHub at https://github.com/piyushrajyadav/formease. The package is distributed under the MIT License, allowing free use in both commercial and open-source projects.

Community Guidelines

FormEase welcomes community contributions and follows standard open-source development practices. The project repository includes:

- Comprehensive documentation with API reference and framework integration examples
- Issue templates for bug reports and feature requests
- Contributing guidelines for pull requests and code style
- Interactive demo showcasing all features with live code examples
- Detailed changelog documenting version history and breaking changes

Users can report issues or request features through the GitHub issue tracker at https://github.com/piyushrajyadav/formease/issues. The project maintainer commits to reviewing and responding to issues within 48 hours. Pull requests are reviewed based on code quality, test coverage, and alignment with project goals.

Acknowledgements

This project was independently developed and maintained by Piyush Yadav. Special thanks to the open-source community for feedback during development and to the maintainers of the Jest testing framework and Rollup bundler used in the build pipeline.

References

Awad, A. (2023). *VeeValidate: Form Validation for Vue.js*. https://vee-validate.logaretm.com/.

Luo, B. (2023). *React Hook Form: Performant, flexible and extensible forms with easy-to-use validation*. https://react-hook-form.com/.

MDN Web Docs. (2025a). *Client-side form validation*. https://developer.mozilla.org/en-US/docs/Learn/Forms/Form_validation.

DRAFT

MDN Web Docs. (2025b). *Web Forms — HTML: HyperText Markup Language*. https://developer.mozilla.org/en-US/docs/Learn/Forms.

Palmer, J. (2023). *Formik: Build forms in React, without the tears*. https://formik.org/.

W3C Web Accessibility Initiative. (2018). Web Content Accessibility Guidelines (WCAG) 2.1. *W3C Recommendation*.

W3C Web Accessibility Initiative. (2021). *WAI-ARIA Authoring Practices 1.2*. https://www.w3.org/TR/wai-aria-practices-1.2/.

DRAFT